Case 02 · Architecture: Fire-Sale Cascade Simulator

Chapter 3 (Architecture) of the Magshimim project-book template: overview, mapping to the spec, and data design and information entities. Three sections not in the template were added for the comparison stage: architectural patterns and trade-offs, failure scenarios and build order, and a closure check.

Overview

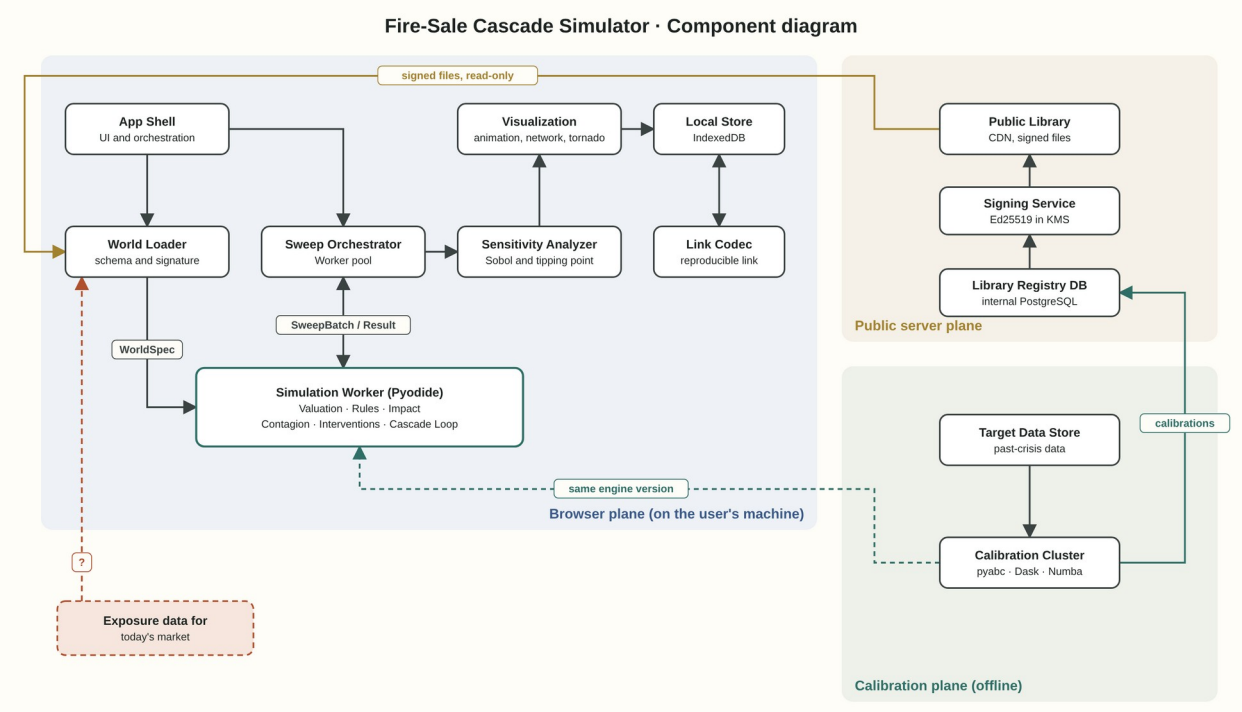
The principle that organises the whole structure

The threat model established that balance-sheet data a user loads are sensitive, so computation runs on the user's machine. This yields a split into three planes, and every component belongs to exactly one of them:

- **The browser plane:** everything that touches a user's world and results. Runs on their computer and sends no data out.
- **The public server plane:** a library of signed static files (worlds, rules, calibrations). Accepts read requests only.
- **The calibration plane:** an offline compute cluster that produces the calibrations and publishes them to the public server.

There is no channel through which information flows from the browser to the server. This is not just policy: it is enforced by a CSP header, which blocks in the browser any request that is not a read from our own server.

Component diagram



The components

- **App Shell (browser plane):** the application itself. Manages screens, receives user input, and hands work to components running in the background. Written in TypeScript, without a heavy framework.
- **World Loader:** loads a world from one of three sources (an official world from the library, a set of distributions, or a private file), validates it against the schema, and checks the signature of an official world. Produces a WorldSpec and starting balance sheets.
- **Simulation Worker:** a Web Worker running Pyodide with NumPy, with the simulation engine inside. Six sub-components: Valuation (revaluing balance sheets at current prices), Rules Interpreter (runs the response rules loaded as data), Market Impact (turns sales into price moves), Contagion (direct contagion and DebtRank), Interventions (fires interventions by trigger), and Cascade Loop (the loop that ties them together and returns a CascadeResult).
- **Sweep Orchestrator:** splits a sweep of hundreds of worlds across several Simulation Workers in parallel (by the computer's core count), collects results, and passes them on for analysis.
- **Sensitivity Analyzer:** computes Sobol indices, percentiles and tipping points from sweep results.
- **Visualization:** the cascade animation (Canvas), the network map (d3-force), the tornado chart (SVG).
- **Link Codec:** encodes and decodes a run into a reproducible link (in the URL fragment, which is not sent to the server).
- **Local Store:** IndexedDB. History of runs, sweeps and private worlds.
- **Public Library (server plane):** a CDN with static files: agent types, rule sets, official worlds and their signatures, calibrations and fit reports, and the public verification key.
- **Library Registry DB:** a PostgreSQL in which the library is managed before publication. Not reachable from outside: it exports files to the public catalog.
- **Signing Service:** signs every official world and calibration file before publication. The private key lives in a KMS.
- **Calibration Cluster (calibration plane):** Dask with pyabc and Numba. Runs the same simulation engine (exactly the same version) against target data from past crises, and produces a parameter distribution and a fit report.
- **Target Data Store:** the target data for calibration, with a source for every number (Bank of England, Fed and SVB reports).

Component table: role and interfaces

Component	Role	Incoming interfaces	Outgoing interfaces
App Shell	Screens, user input, orchestration	User actions, reproducible link	Requests to World Loader, Sweep Orchestrator and Visualization
World Loader	Loading, schema validation, signature check	Library world file, private file, distribution definition	WorldSpec and starting balance sheets
Simulation Worker	Full cascade on one world	RunRequest (world, shock, interventions,	CascadeResult

Component	Role	Incoming interfaces	Outgoing interfaces
		seed)	
Sweep Orchestrator	Splitting a sweep across Workers and collecting	SweepSpec	A set of CascadeResults for analysis
Sensitivity Analyzer	Sobol, percentiles, tipping points	Results and parameters	SweepResult
Visualization	Animation, network, tornado, accessible caption	CascadeResult, SweepResult, ExposureGraph	Display and downloadable image
Link Codec	Encoding and decoding a run as a link	RunLink	URL with fragment
Local Store	Local storage	Results and private worlds	Retrieval for history
Public Library	Serving signed static files	GET requests only	JSON, Parquet and signature files
Library Registry DB	Managing the library before publication	Updates from the project team, calibrations	Export to files
Signing Service	Signing official content	Canonical file	Ed25519 signature
Calibration Cluster	Calibration against past crises	Target data, priors, engine	Parameter distribution, fit report
Target Data Store	Target data with sources	Manual entry from public reports	Target data for calibration

Mapping to the spec

Spec feature	Components involved	Order of operations and why
1. World building and agent library	App Shell → World Loader → Public Library	The signature check happens before the world is passed to the engine, so a world that fails it is never marked "official"
2. Balance sheet and valuation	Simulation Worker (Valuation)	Runs at the start of every round, before the rules, because rules decide on current economic equity
3. Response rules	Public Library (rule sets) → World Loader → Simulation Worker (Rules Interpreter)	Rules are loaded as data and compiled once into vector operations, not every round
4. Price impact	Simulation Worker (Market Impact)	Runs after interventions, so central-bank absorption reduces sales before they move the price
5. Cascade engine	Simulation Worker (Cascade Loop)	The loop itself: valuation, rules, settlement, intervention, price impact, contagion, conservation checks
6. Network and contagion	World Loader (graph building) → Simulation Worker (Contagion)	DebtRank is computed once before the run (structure), and contagion every round (dynamics)
7. Intervention	App Shell (definition) → Simulation Worker (Interventions)	The announcement affects the impact coefficient from the round it is announced, and purchases from the next round
8. Sweep and sensitivity	App Shell → Sweep Orchestrator	A single run is also routed

Spec feature	Components involved	Order of operations and why
	→ Simulation Workers (in parallel) → Sensitivity Analyzer	automatically to a default sweep, so there is no result without a range
9. Replay and calibration	Target Data Store → Calibration Cluster → Signing Service → Public Library	Calibration does not run at the user. It is published as a signed file, and official worlds point to it
10. View and link	Visualization, Link Codec, Local Store	A link is created only for a non-private world, and the fragment is not sent to the server

Data design and information entities

Information passed between components (not stored)

- **RunRequest, from App Shell or Sweep Orchestrator to a Simulation Worker:** a world identifier or the balance sheets themselves (as a Transferable ArrayBuffer, without copying), shock, list of interventions, seed and engine version.
- **CascadeResult, from a Simulation Worker to the Orchestrator or App Shell:** compressed RoundState arrays (typed arrays) and summary measures.
- **SweepBatch, from the Sweep Orchestrator to a Worker:** a list of parameter vectors from a segment of the Sobol sequence, so each Worker gets a separate, complete portion.
- **ProgressEvent, from Workers to the App Shell:** how many runs have finished, for showing progress and allowing cancellation.

Information that is stored

- **On the public server (static files, read-only):** agent_types, rule_sets, worlds with signatures and sources, calibrations (a Parquet posterior and a JSON fit report), and a public key.
- **In the Library Registry DB (internal):** the same entities in their working version, with change history. This is the table exports come from.
- **In the Calibration Cluster:** target data, calibration runs and posteriors. Internal, published only through the Signing Service.
- **In the browser (IndexedDB):** runs, sweeps, private_worlds. What is stored here never reaches the server.

Deliberate separation

A run's result (CascadeResult) and its link (RunLink) are two separate entities. The link contains only what is needed to reproduce the run, not the result. So even a leaked link reveals only a configuration, never a user's balance-sheet data.

Architectural patterns and trade-offs

Functional Core, Imperative Shell

The simulation engine is a pure function: the same world, the same shock and the same seed always return exactly the same result. It has no access to the network, the clock or outside

randomness. Everything with side effects (reading files, saving, drawing) lives in the shell, in the App Shell and the Local Store.

- **What we gain:** full reproducibility (the reproducible link works only because of this), simple tests, and exactly the same engine in the browser and the calibration cluster.
- **What we pay:** the whole state must be passed to the engine as input and returned as output, which means more memory copying. We reduce this with typed arrays transferred without copying.

Data-Oriented Design (inspired by ECS in game engines)

Instead of an object per agent with methods, every attribute is an array over all agents (Structure of Arrays), and every "system" (valuation, rules, impact) is a function that passes over the arrays. This is how game engines managing millions of entities work.

- **What we gain:** the POC measured 1,000 runs in 0.28 seconds. A simulation with an object per agent would be tens of times slower.
- **What we pay:** code that reads less "naturally". An agent is not an object you can hold, but an index into arrays. That is why the rules interface (feature 3) is written in terms of a single agent, and translated into array operations.

Rules as data (Interpreter pattern)

Response rules are YAML files that are loaded and translated into operations. The engine does not know what an "LDI fund" is; it knows how to apply a condition and an action.

- **What we gain:** an analyst can read and change a rule, and two rule sets can be compared on the same world.
- **What we pay:** the rule language is deliberately limited. A rule it cannot express requires an engine change. And we must make sure the interpreter never runs arbitrary code from a user file (no eval).

Fork-Join in the browser (over a pool of Workers)

A sweep is split by segments of the Sobol sequence across several Workers, and each Worker returns partial results.

- **What we gain:** a sweep of 512 worlds on an 8-core machine runs about 6 times faster.
- **What we pay:** every Worker loads Pyodide separately (about 10 megabytes and a few seconds the first time). So the pool is created once and stays alive for the whole session.

Read-Only Publication

The public server accepts no writes from any user. All its content is created in an internal process, signed, and published as files.

- **What we gain:** a tiny attack surface (no forms, no write API, no exposed database), and everything can be served from a CDN.
- **What we pay:** no sharing between users inside the system. Anyone who wants to share sends a link.

Failure scenarios

What fails	What happens	Why that is acceptable (or not)
The CDN is unavailable	Official worlds do not load. Distribution-based and private worlds work from the local cache	The engine runs locally, so the tool works partially even without a server
A world's signature is invalid	The world loads as "custom", with a prominent warning	A result cannot be presented as official without a valid signature
A Worker crashes mid-sweep	The Orchestrator notices and reruns the segment on another Worker	Runs are deterministic, so a rerun gives the same result
The cascade does not converge after 100 rounds	The result is marked "did not converge" and is not shown as a number	A spiral that does not end is a finding, not a bug, and must be seen as one
A conservation check fails	The run stops with an error and details	An error is better than a wrong result that looks plausible
Calibration fails to reproduce a target	The report marks it in red, and the official world publishes this	The limitation is visible, not hidden

Build order

- **Sprint 1, minimal engine:** Valuation, Cascade Loop and two agent types (LDI and bank) in plain Python, with conservation checks. Goal: reproduce the POC in the new structure.
- **Sprint 2, rules and impact:** Rules Interpreter with YAML, the three impact models, and all five agent types.
- **Sprint 3, browser:** move the engine to Pyodide inside a Worker, World Loader, and a basic App Shell with a yield chart.
- **Sprint 4, sweep:** Sweep Orchestrator, Sensitivity Analyzer, and the tornado chart.
- **Sprint 5, contagion and intervention:** Contagion, DebtRank, and the four kinds of intervention.
- **Sprint 6, library and calibration:** Library Registry, Signing Service, Calibration Cluster, and the first two official worlds.
- **Sprint 7, polish:** animation, network map, reproducible link, Local Store, CSP and accessibility.

Closure check

The closure check asks a simple question: can the system keep its central promise end to end, using only components inside its own boundaries? The project's central promise, from ideation: to see a crisis that arrives slowly, before it arrives fast. In other words, to say something about the market as it is today.

Component or input	Inside the boundaries?	Does the promise depend on it?
The simulation engine (all sub-components)	Yes	Yes, and fully under control
Sweep and sensitivity	Yes	Yes, and fully under control

Component or input	Inside the boundaries?	Does the promise depend on it?
Official worlds of past crises	Yes (calibrated from public reports)	Only for replaying the past
Calibration target data	Yes (from public reports)	Only for calibration
The distribution of leverage and exposures in today's market	No	Yes, and it is the decisive input

The last row is the problem. The POC showed, and the sweep in feature 8 will measure it again on every run, that between average leverage of 4 and 5.5 the outcome moves from "calm" to "near-certain spiral". The system does not and cannot produce this figure: it can only guess a range, or receive it from an outside source. In the component diagram it is marked as a dashed box outside the boundaries: "exposure data for today's market". No component fills it.

What the system does promise, fully and within its boundaries: "if leverage and exposures are X, this is what happens, and this is how sensitive the answer is to X". That is a conditional answer. It is very useful for learning and for replaying past crises. But on the question that started the project, what is the state of the market now, its answer passes through a box it does not have.